



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Binary Trees – Implementation



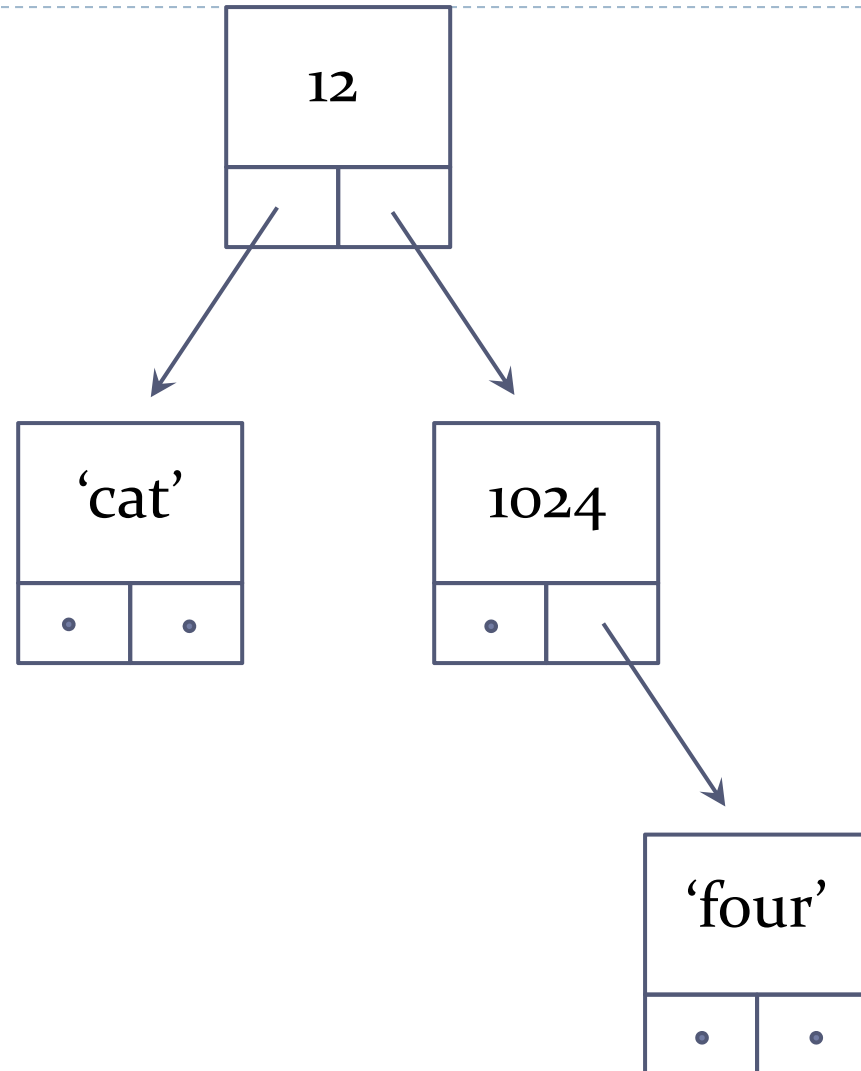
Agenda & Reading

- ▶ **Agenda**
 - ▶ BinaryTree class
- ▶ **Online Coursebook:**
 - ▶ <https://onlinetextbook.auckland.ac.nz/19-binary-trees/>



The BinaryTree Class

- ▶ We will define a simple **BinaryTree** class which will allow us to represent binary trees as in the following diagram:





BinaryTree Class - Constructor

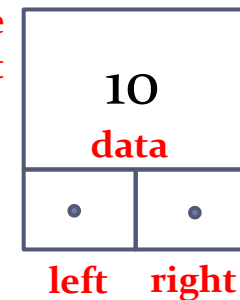
- ▶ Let's start with the initialisation function:

```
class BinaryTree:
```

```
    def __init__(self, data, left=None, right=None):  
        self.data = data  
        self.left = left  
        self.right = right
```

```
a_tree = BinaryTree(10)
```

BinaryTree
object



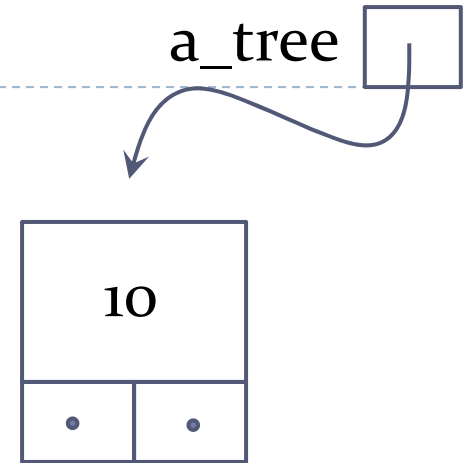
The references **left** and **right** can point to other `BinaryTree` objects



BinaryTree Class – Adding Children

- ▶ How do we add a child to this tree?

`a_tree = BinaryTree(10)` →

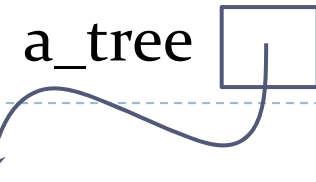
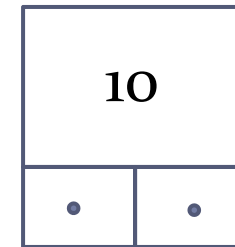




BinaryTree Class – Adding Children

- ▶ How do we add a child to this tree?

```
a_tree = BinaryTree(10)
```

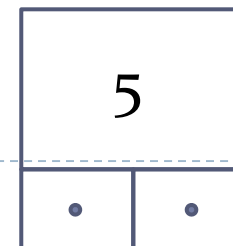
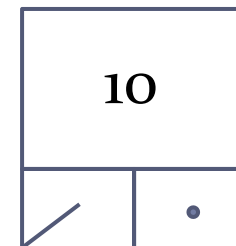


```
a_tree.insert_left(5)
```



If there is no existing left child...

```
self.left = BinaryTree(5)
```

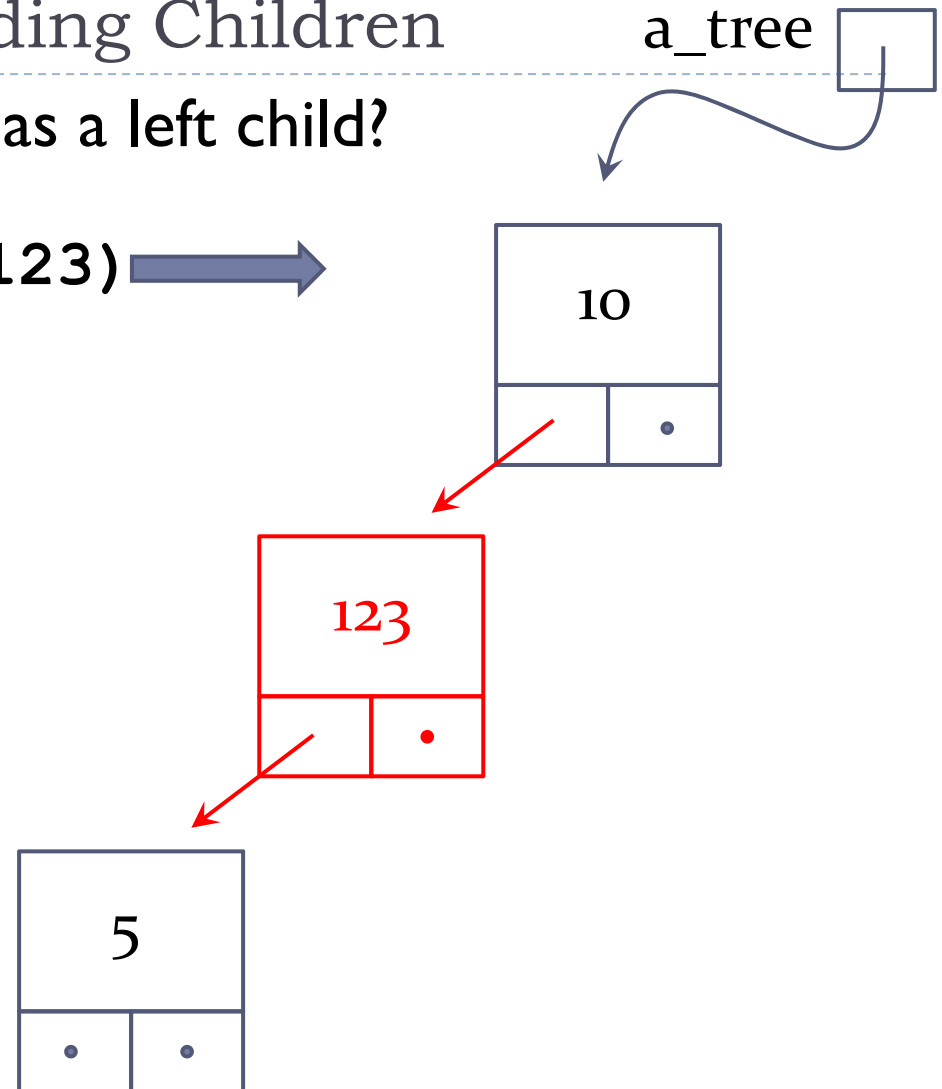




BinaryTree Class – Adding Children

- ▶ But what if **a_tree** already has a left child?

`a_tree.insert_left(123)` →



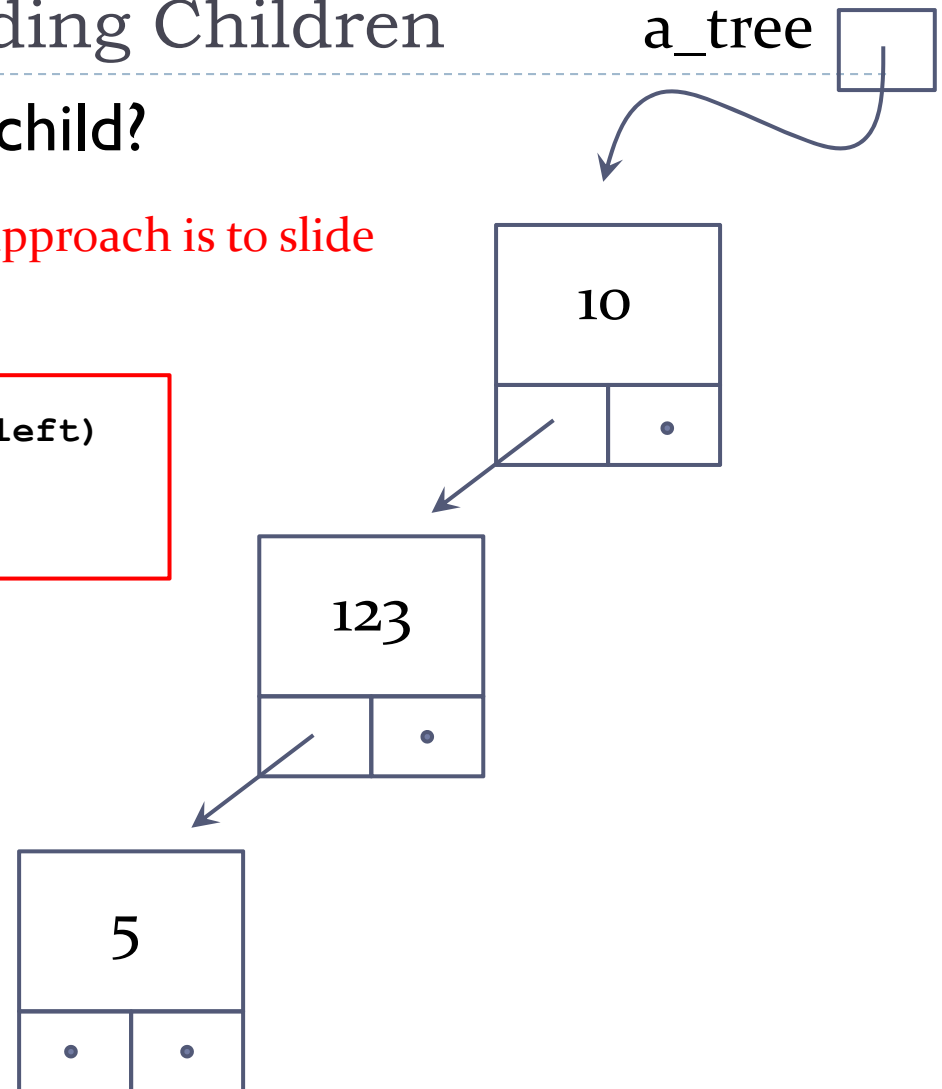


BinaryTree Class – Adding Children

► But what if a already has a left child?

If there is an existing left child, the simplest approach is to slide the existing subtree down...

```
subtree = BinaryTree(new_data, left=self.left)
self.left = subtree
```





BinaryTree Class

► The BinaryTree class so far:

```
class BinaryTree:

    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right

    def insert_left(self, new_data):
        if self.left == None:
            self.left = BinaryTree(new_data)
        else:
            subtree = BinaryTree(new_data, left=self.left)
            self.left = subtree

    def insert_right(self, new_data):
        if self.right == None:
            self.right = BinaryTree(new_data)
        else:
            subtree = BinaryTree(new_data, right=self.right)
            self.right = subtree
```



BinaryTree class

► The BinaryTree class so far:

- We can also add basic methods for getting and setting the data and for getting the references to left and right subtrees:

```
def get_left(self):  
    return self.left
```

```
def get_right(self):  
    return self.right
```

```
def set_data(self, data):  
    self.data = data
```

```
def get_data(self):  
    return self.data
```



Example

- Draw the tree created by the following code:

```
a_tree = BinaryTree(10)
```

```
a_tree.insert_left(5)
```

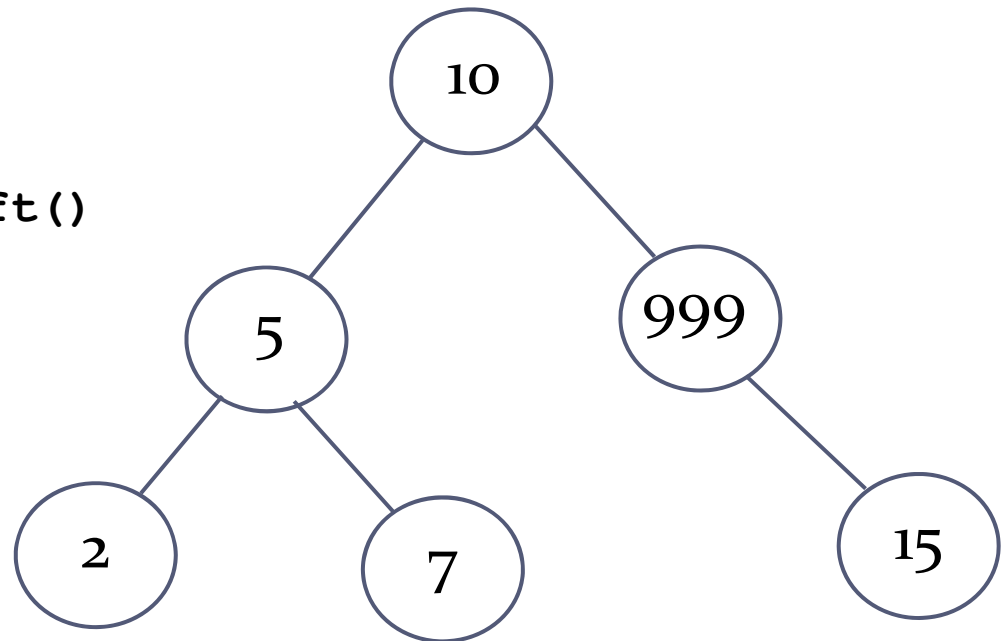
```
a_tree.insert_right(15)
```

```
a_tree.insert_right(999)
```

```
left_subtree = a_tree.get_left()
```

```
left_subtree.insert_left(2)
```

```
left_subtree.insert_right(7)
```





Exercise 3

- Draw the tree created by the following code

```
tree = BinaryTree(5)
tree.insert_left(8)
tree.insert_right(2)
right = tree.get_right()
left = tree.get_left()
left.insert_left(7)

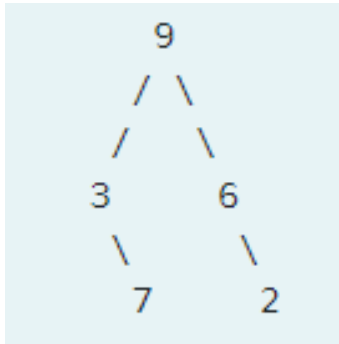
value = left.get_data()
left.set_data(value + 4)

right.insert_right(3)
right.insert_left(6)
```



Exercise 4

- ▶ Define a function called `create_a_tree()` which builds and returns the following binary tree:



```
def create_a_tree():  
    tree = BinaryTree(9)
```